

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА
ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 5

з навчальної дисципліни

«Технології розробки корпоративних Web-додатків»

Виконав:

Студент групи 403-ТН

Солонецький Роман Миколайович

Перевірив:

Мізік Андрій Олегович

Полтава 2026

Тема: Вебдодаток для відстеження звичок (HabitTracker)

Для звіту використовую вже майже готовий проєкт до дипломної роботи.

Структура проєкту та сервісів API

Клієнт-серверна архітектура проєкту побудована за принципом чіткого розділення відповідальності (Separation of Concerns).

- На сервері реалізовано шар **Сервісів** (habitService.js), який інкапсулює бізнес-логіку, віддаляючи її від контролерів (маршрутів).
- Використовуються **DTO (Data Transfer Objects)**, наприклад HabitDto.js, для форматування даних перед відправкою на клієнт.
- Взаємодія фронтенду з бекендом винесена в окремі API-модулі (або здійснюється через налаштований екземпляр axios), що забезпечує єдину точку входу для всіх HTTP-запитів.

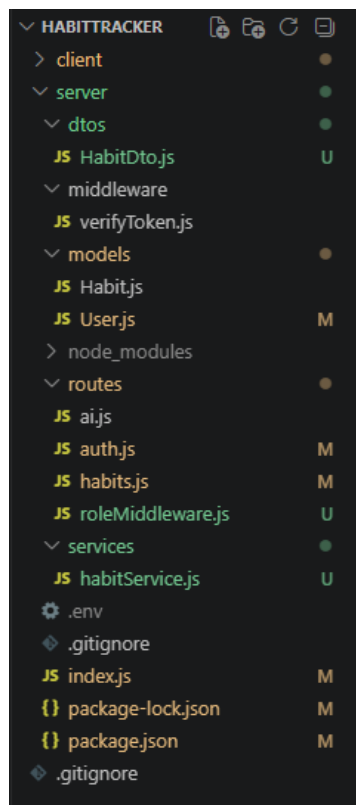


Рис 1 – Дерево файлів проєкту

Реалізація автентифікації та захищених маршрутів

У додатку реалізовано механізм захисту маршрутів на основі JWT.

Після успішної авторизації клієнт отримує токен і зберігає його в localStorage.

Усі наступні запити до API автоматично включають цей токен у заголовок auth-token (або Authorization).

На рівні React-роутингу створено компонент-обгортку (Private Route), який перевіряє наявність токена. Якщо токен відсутній, неавторизованого користувача автоматично перенаправляє на сторінку /login, перекриваючи доступ до головного дашборду зі звичками.

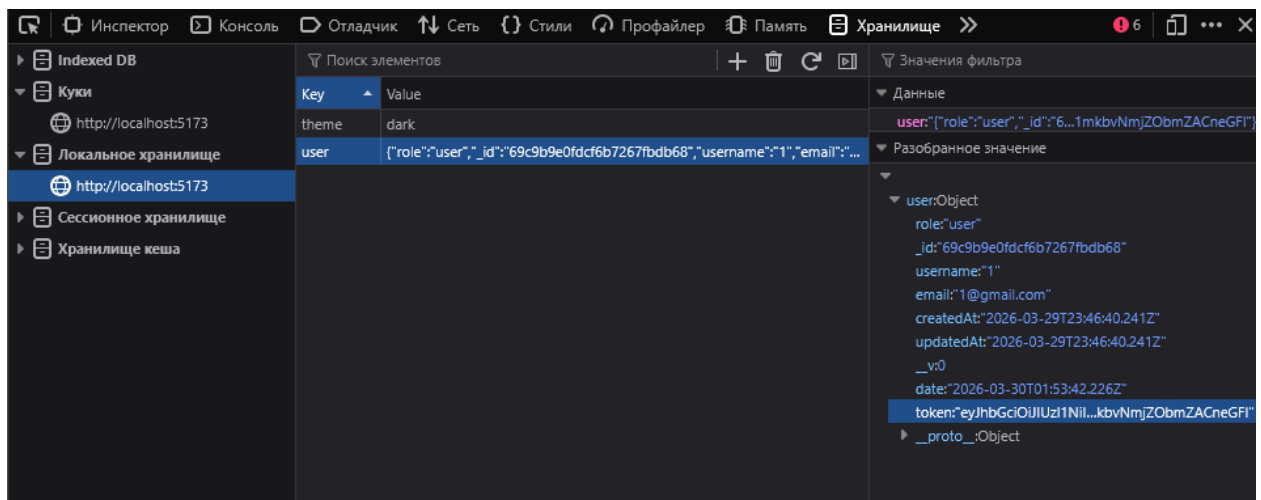


Рис 2 – Отримано токен

Реалізація повного CRUD інтерфейсу

Інтерфейс користувача підтримує повний набір операцій CRUD для управління звичками:

- **Create:** Форма додавання нової звички (запит POST /api/habits).
- **Read:** Динамічне виведення списку звичок поточного користувача при завантаженні сторінки (запит GET /api/habits).
- **Update:** Інтерактивна можливість відмічати виконання звички, що оновлює її статус у базі даних (запит PUT /api/habits/:id/toggle).
- **Delete:** Кнопка видалення непотрібної звички із миттєвим оновленням UI без перезавантаження сторінки (запит DELETE /api/habits/:id).

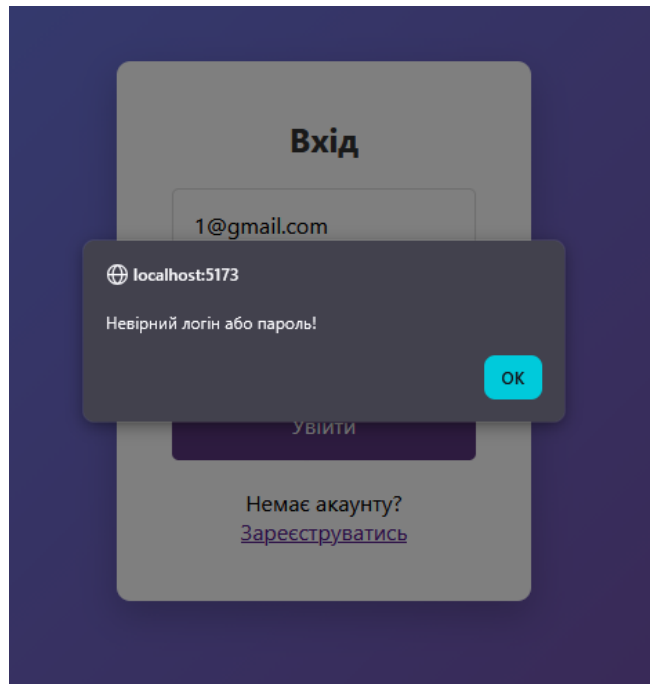


Рис 4 – Помилка при авторизації

Спінери: Під час виконання асинхронних запитів до API (наприклад, логін або завантаження списку) активується стан завантаження (спінер або блокування кнопок), що інформує користувача про процес і запобігає повторним випадковим клікам.

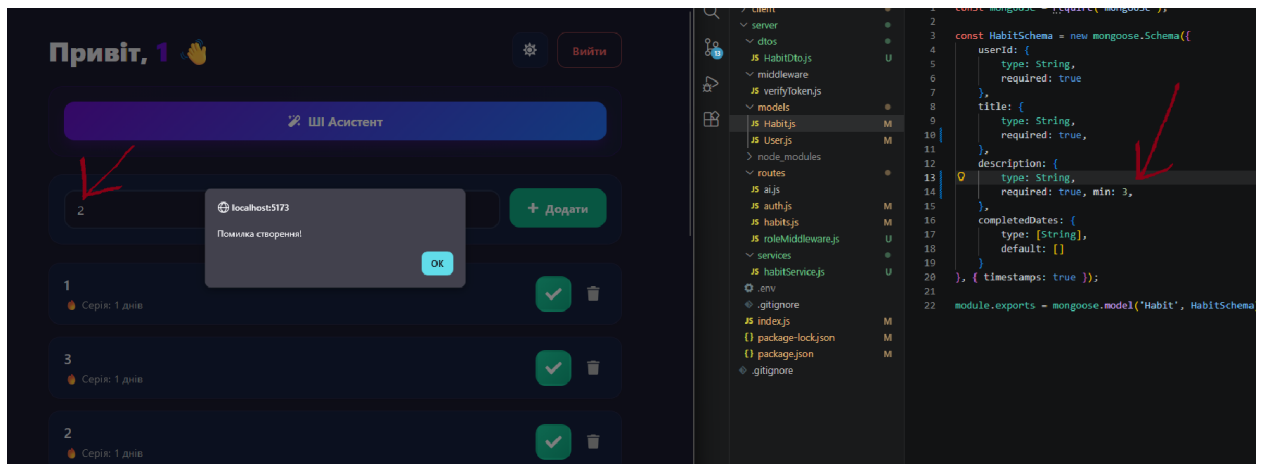


Рис 5 – Помилка введенні не вірного заголовку звични (для прикладу зроблено мінімум 3 символа)

Контрольні питання

1. Що таке SPA (Single Page Application) і чим воно відрізняється від класичних МРА? Які переваги та недоліки?

SPA оновлює сторінку динамічно без перезавантаження (швидше працює). МРА при кожному кліку вантажить нову сторінку з сервера.

2. Що таке Virtual DOM (у контексті React/Vue) і навіщо він потрібен?

Копія інтерфейсу в пам'яті. React шукає зміни в ній і оновлює на екрані лише те, що змінилося.

3. Як працює Routing на клієнті? Чим History API відрізняється від звичайних посилань?

Працює через HTML5 History API. URL змінюється, але реального запиту на сервер не відбувається — React просто малює інший компонент.

4. Де безпечніше зберігати JWT токени на клієнті: localStorage, sessionStorage чи HttpOnly Cookies? Обґрунтуйте.

HttpOnly Cookies: Найбезпечніший варіант, ці куки неможливо прочитати через JavaScript, що повністю захищає токен від XSS-атак.

localStorage: Менш безпечні, оскільки доступні для JavaScript (вразливі до XSS). Однак вони часто використовуються в навчальних та невеликих проєктах через простоту реалізації.

5. Що таке CORS? Чому браузер блокує запити до іншого домену і як це вирішити?

Cross-Origin Resource Sharing - це механізм безпеки браузера, який блокує запити веб-сторінки до іншого домену, порту або протоколу (наприклад, коли фронтенд на порту 5173 звертається до бекенду на порту 5000). Бекенд-сервер повинен явно дозволити такі запити, відправивши спеціальні HTTP-заголовки. У Node.js/Express це робиться одним рядком за допомогою: `app.use(cors())`.

6. Поясніть концепцію State Management (управління станом). Коли достатньо локального стану компонента, а коли потрібен глобальний (Redux/Context/Pinia)?

Локальний стан (useState у React) достатній для даних, які потрібні лише всередині одного компонента.

Глобальний стан (Redux, Zustand, Context API) потрібен тоді, коли однакові дані використовуються в багатьох непов'язаних компонентах, щоб уникнути "прокидання" пропсів через усі рівні (prop drilling).

Вебдодаток залитий на сервер

Готовий проект вже залитий на гіт (перший варіант проекту дипломної роботи):

<https://github.com/1ssue/habit-tracker>

та підключений до vercel.com та render.com.

Спершу очікуємо увімкнення БД:

<https://habit-tracker-wtyx.onrender.com/>

Потім можна авторизуватись з боку клієнта:

<https://habit-tracker-ten-sepia.vercel.app/>

Можна використати готовий профіль:

admin@gmail.com – 123456 або створити свій.

Якщо виникає помилка після авторизації потрібно видалити у посиланні /login та перезавантажити (з часом праную вирішити цю проблему).